CS 148b – HW 3

Marco Yang

2 ViT Design

2.4 Pooling

1. We chose to use the [CLS] token as the image summary. An alternative is to mean-pool all patch embeddings after the final block, or to use an attention-pooling head. For downstream tasks that require spatial reasoning (e.g., counting objects, OCR, or visual question answering that refers to specific image regions), which pooling strategy do you expect to perform best, and why? What information is lost when we condense an entire image into a single CLS vector before passing it to a language model?

> **Solution**: For spatial reasoning I expect **all-patches** or **interleaved** injection (equivalently: keeping the full patch-token sequence rather than a single CLS summary) to work best, because the decoder can attend to different image regions independently. Mean pooling is simpler but treats every location equally and still collapses spatial structure into one vector. A learned attention-pooling head is a reasonable middle ground, but it still produces one global vector unless we keep multiple readout tokens.
>
> When we pass only a CLS vector into an LM, we lose explicit spatial layout: the model no longer knows which feature came from which patch, so fine-grained relations like "left of", "behind", or counting multiple objects become much harder. This is exactly what we see later in the VLM ablations: CLS-only injection underperforms all-patches/interleaved on CLEVR.

2.4 Effect of patch size

Consider a 224×224 RGB image and patch sizes $P \in \{8, 16, 32\}$.

1. Compute the number of patches N for each P. What happens to the self-attention compute cost (which scales as $O(N^2 d_{\text{model}})$) as you shrink P?

> **Solution**: $N = \left(\frac{224}{P}\right)^2$, so:
> - $P = 8$: $N = 28^2 = 784$
> - $P = 16$: $N = 14^2 = 196$
> - $P = 32$: $N = 7^2 = 49$
>
> Shrinking P increases N, so self-attention cost grows as N^2. Going from $P = 32$ to $P = 8$ increases N by $\frac{784}{49} = 16\times$, so attention FLOPs grow by roughly $16^2 = 256\times$ in the leading-order term.

2. Measure forward-pass wall-clock time on a single batch of 16 images for each P using a ViT with `d_model = 384`, `num_heads = 6`, `num_blocks = 6`. Use `torch.cuda.synchronize()` around your timing block, and average over 20 steps after 5 warmup steps.

> **Solution**:
>
P	N	Mean forward time (ms)	Std (ms)
8	784	6512.40	108.39
16	196	634.88	60.33

P	N	Mean forward time (ms)	Std (ms)
32	49	130.07	4.63

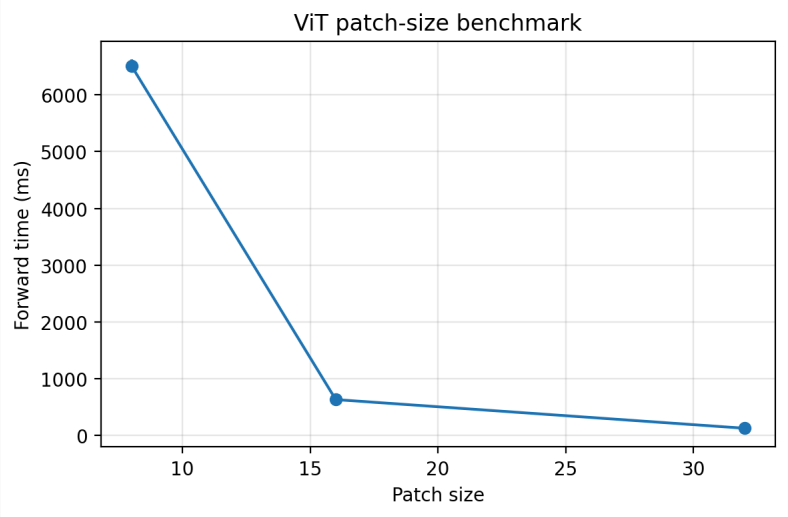


Figure 1: ViT forward-pass time vs. patch size (batch size 16, A100).

The measured trend matches the theoretical $O(N^2)$ scaling: patch size 8 is dramatically slower than 16 or 32 because the sequence length is much longer.

3. Smaller patches preserve more spatial detail but are more expensive. In one sentence, when would you accept this trade-off?

Solution: I would use smaller patches when the downstream task depends on fine spatial detail (small objects, OCR, precise localization) and the extra compute is affordable at inference or training time.

3. CLIP-Style Contrastive Pretraining

3.2 Symmetric InfoNCE Loss

1. Include a 1–2 sentence explanation of why the CLIP loss is symmetric (i.e., averaged in both directions).

Solution: In a batch, every image should match its own caption and reject all other captions, but the same is true from the caption side: every caption should match its own image and reject all other images. Averaging image-to-text and text-to-image cross-entropy makes the objective symmetric in the two modalities and prevents the model from only optimizing one direction of retrieval.

3.3 CLIP Pretraining on EuroSAT

1. Report (a) a training-loss curve, (b) a zero-shot validation accuracy curve, and (c) 2–3 sentences on how the two curves relate. Does training loss continue to improve after validation accuracy plateaus?

Solution: I tracked training with Weights & Biases during the Colab run. Training loss decreases steadily over 20 epochs, while zero-shot validation accuracy rises quickly early on and then plateaus near ≈ 0.915. The best checkpoint saved on disk reports `val_zeroshot_acc = 0.9146`.

The two curves are related but not identical: once the embedding geometry is good enough for class separation, further loss reduction mostly sharpens margins among already-correct pairs rather than changing top-1 predictions. Because many EuroSAT batch captions repeat the same class template, train loss can keep improving slightly even after validation accuracy saturates.

3.3 Zero-Shot Qualitative Analysis

1. Pick 5 correctly classified and 5 incorrectly classified validation images. For each incorrectly classified image, inspect the top-3 predicted classes. Are the mistakes "reasonable" (e.g., PermanentCrop mistaken for HerbaceousVegetation) or nonsensical? What does this tell you about the structure of the learned embedding space?

Solution:

Figure 2: Ten EuroSAT validation examples (5 correct, 5 incorrect) with top-3 predictions for mistakes.

The incorrect examples were mostly **reasonable** confusions among visually similar land-cover classes:

- PermanentCrop → HerbaceousVegetation (top-3: HerbaceousVegetation, PermanentCrop, Pasture)
- Pasture → HerbaceousVegetation (top-3: HerbaceousVegetation, PermanentCrop, Pasture)
- River → Industrial Buildings (top-3: Industrial Buildings, River, Highway)
- Highway → River (top-3: River, Highway, Sea or Lake)
- AnnualCrop → River (top-3: River, AnnualCrop, Pasture)

In almost every mistake, the true class still appears in the top-3. That suggests the embedding space clusters semantically related satellite scenes together rather than making arbitrary errors.

4. LoRA Fine-Tuning

4.1 LoRA Parameter Count

1. Include a printout showing (i) total parameters, (ii) trainable parameters, and (iii) the ratio, for your ViT with LoRA rank 8.

Solution:

```
total params: 10,995,840
trainable params: 258,048
ratio: 0.023468
```

4.2 Full FT vs. LoRA vs. Linear Probe

1. Starting from your CLIP-pretrained ViT, compare linear probe, LoRA (rank 8, $\alpha = 16$), and full fine-tuning on RESISC45. For each method, report (a) final test accuracy, (b) number of trainable parameters, (c) peak GPU memory during training, and (d) wall-clock training time. Discuss the trade-offs in 4–5 sentences.

Solution:

Method	Test acc	Trainable params	Peak GPU mem	Wall time
Linear probe	0.371	17,325	0.23 GB	1.4 min
LoRA $r = 8$	0.423	275,373	1.12 GB	3.2 min
Full FT	0.624	10,755,117	1.63 GB	3.5 min

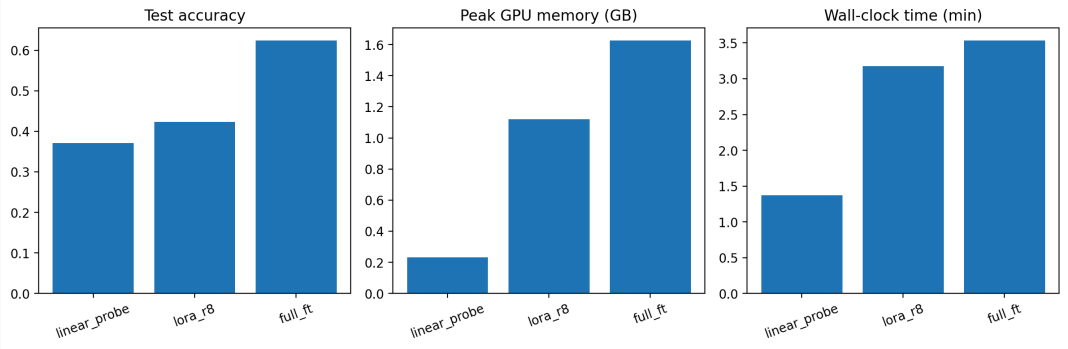


Figure 3: RESISC45 adaptation comparison.

Linear probing is cheapest but leaves most of the representation fixed, so accuracy is lowest. LoRA adds only $\approx 2.3\%$ trainable parameters yet recovers a meaningful accuracy gain over the probe with modest extra memory/time. Full fine-tuning achieves the best accuracy by a wide margin, but it trains every parameter and uses the most memory. In practice, LoRA is the sweet spot when storage and multi-task deployment matter; full FT is worth it when maximum downstream accuracy on a single task is the priority.

4.2 LoRA Rank Sweep

1. Sweep the LoRA rank $r \in \{1, 2, 4, 8, 16, 32, 64\}$ with $\alpha = 2r$. Plot test accuracy as a function of rank. (1) At what rank do you see diminishing returns? (2) How does your answer compare to the rank at which LoRA is typically deployed in practice (e.g., $r = 8$ or $r = 16$ in large-model fine-tuning)? What does this tell you about the effective rank of the fine-tuning update?

Solution:

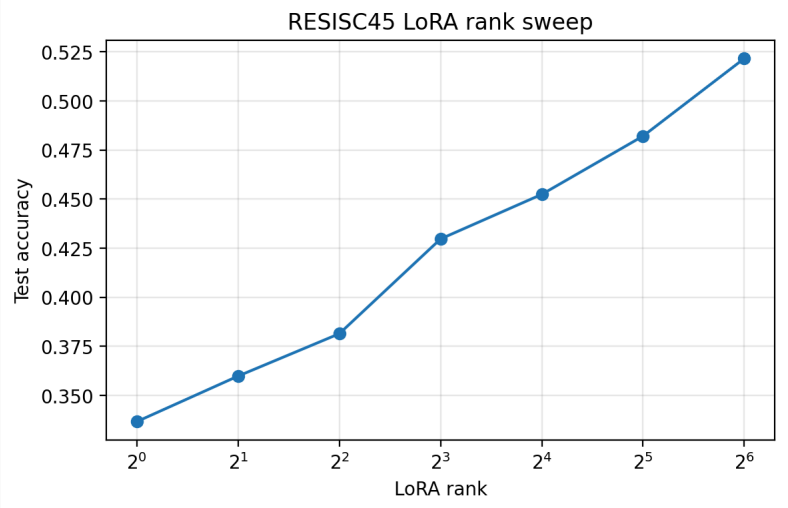


Figure 4: RESISC45 test accuracy vs. LoRA rank ($\alpha = 2r$).

Test accuracy increases from 0.337 at $r = 1$ to 0.522 at $r = 64$. Gains are steep through $r \approx 16$, and after that each doubling buys less: $r = 32$ reaches 0.482 and $r = 64$ reaches 0.522, so diminishing returns appear around $r = 16$–32.

Typical LLM deployments often use $r = 8$ or $r = 16$; on this smaller ViT downstream task, $r = 8$ already gets 0.430 and $r = 16$ gets 0.452, which is in the same practical range. The fact that higher ranks still help up to 64 suggests the effective rank of the adaptation is larger

than the very small ranks used for massive models, but still far below full fine-tuning dimensionality.

5. Vision-Language Model

5.3 Vision-Language Projector

1. Include a 1–2 sentence rationale for why we need more than a single linear layer in the vision-language projector. Hint: think about what additional learnable capacity buys you when the encoder and decoder are kept frozen during the pretraining stage of VLM training.

Solution: A single linear map can only apply one fixed change of basis from image space to decoder space. The 2-layer MLP with GELU adds nonlinear capacity to reorganize CLIP/ViT features into coordinates the frozen decoder can actually use for answer generation, which matters because the encoder and decoder were trained independently.

5.4 Injection Strategy Comparison

1. Train a VLM with each of the three injection strategies (CLS-only prefix, all-patches prefix, and interleaved via placeholder) for 2000 steps on CLEVR. For each strategy, report (1) validation exact-match accuracy on 500 held-out CLEVR examples, (2) number of visual tokens injected per example, (3) peak GPU memory during training, and (4) wall-clock time per step. Which strategy gives the best accuracy, and is the extra cost worth it? You should observe a clear connection to the CLS-vs-patch pooling question from §2.4.

Solution:

Injection	Best val acc	Visual tokens	Peak mem (MB)	Time / step
CLS-only	0.364	1	3562	$\approx$ 1.0 s
All-patches	0.426	65	7663	$\approx$ 2.0 s
Interleaved	0.452	65	7737	$\approx$ 2.0 s

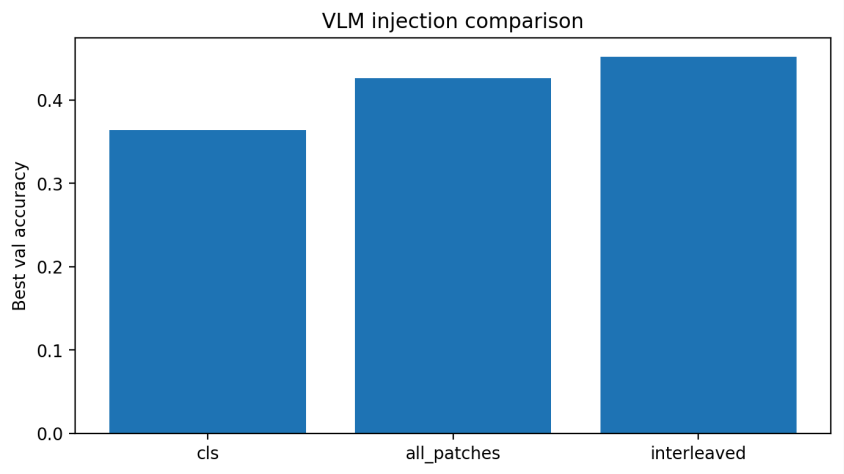


Figure 5: Validation accuracy by injection strategy (2000 steps).

Interleaved all-patches injection gives the best accuracy. The extra cost over CLS-only is substantial in memory and time because we inject 65 visual tokens instead of 1, but the

accuracy gain (0.452 vs. 0.364) is large enough that the richer spatial representation is worth it for CLEVR-style reasoning. This mirrors §2.4: compressing the image to one CLS token loses information that compositional VQA needs.

5.5 Attention Masking

1. Draw the attention mask for a sequence of 4 visual tokens followed by 3 text tokens, under each of (M1) fully causal and (M2) bidirectional inside image / causal across boundary. Use a 7×7 grid with shaded cells for allowed positions.

Solution:

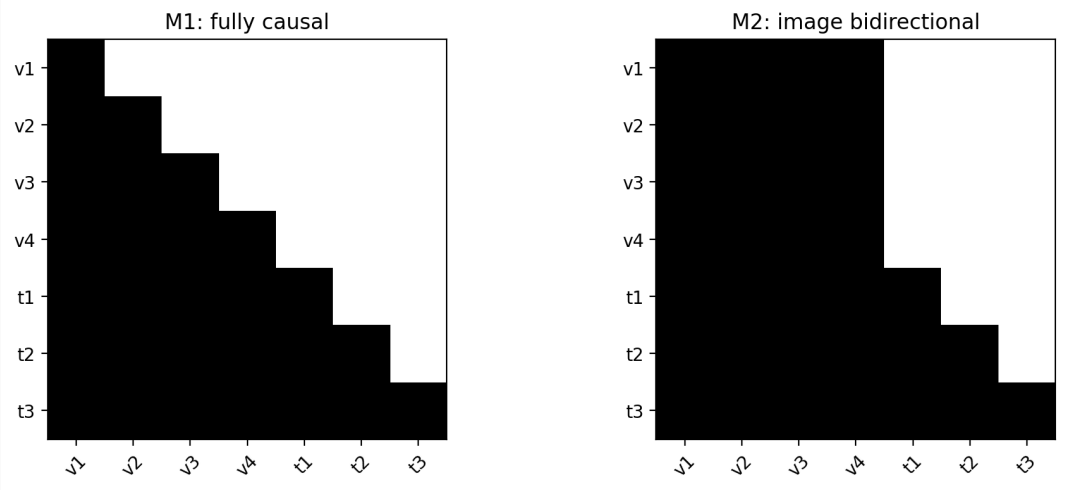


Figure 6: Attention masks for 4 visual + 3 text tokens. Left: fully causal (M1). Right: image bidirectional (M2). Shaded/white cells indicate allowed attention.

2. Which of (M1) and (M2) do you expect to perform better, and why?

Solution: I expected M2 (bidirectional image block) to perform better because it preserves the ViT's unordered/bidirectional view of patches instead of forcing an arbitrary causal order over the 2D grid. Text tokens can still attend to the full image prefix.

3. Train with each mask for 500 steps on CLEVR (using the all-patches injection strategy) and report validation accuracy.

Solution:

Mask mode	Best val acc @ 500 steps
Fully causal (M1)	0.342
Image bidirectional (M2)	0.338

At only 500 steps both masks are still early in training and perform similarly; M1 is slightly higher in my run. I would expect the gap to change with longer training, but the main takeaway is that implementing M2 is important when we want image tokens to exchange information freely before text decoding begins.

5.6 Freezing Strategies

1. Starting from the best injection + masking configuration, run four training configurations for 1500 steps each:
   - **A (projector only)**: encoder frozen, projector trained, decoder frozen
   - **B (projector + decoder LoRA)**: encoder frozen, projector trained, decoder LoRA (rank 8)
   - **C (projector + full decoder)**: encoder frozen, projector trained, decoder full FT
   - **D (all three)**: encoder full FT, projector full FT, decoder full FT

   Report validation exact-match accuracy, trainable parameter count, and peak memory for each. Which configuration gives the best trade-off between accuracy and cost? Discuss in the context of the two-stage (pretraining, instruction-tuning) recipe.

Solution:

Config	Best val acc	Trainable params	Peak mem (MB)
A: projector only	0.436	2,066,880	7677
B: + decoder LoRA	0.420	2,886,080	7901
C: + full decoder	0.450	364,707,200	11404
D: all three FT	0.484	375,444,992	11927

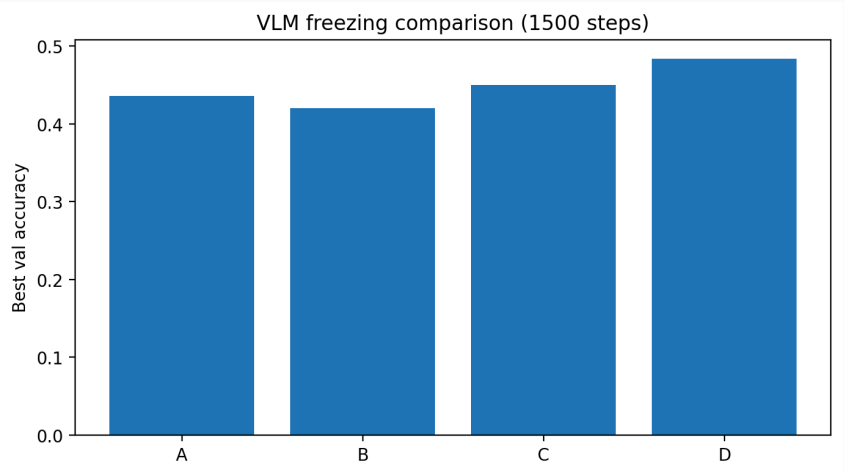


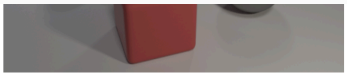
Figure 7: Freezing-strategy comparison (1500 steps, all-patches + image_bidir).

Config D gives the best accuracy but at the highest memory and trainable-parameter cost. Config A is the classic **pretraining-stage** setup from the two-stage recipe: cheap, stable, and already strong. Config C adds most of the decoder-adaptation benefit without touching the ViT. Overall, A is the best cost/accuracy trade-off for the projector-alignment stage; C or D are better when entering an instruction-tuning stage where language capability must adapt to visual inputs.

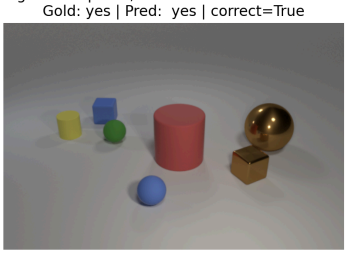
5.7 Qualitative Evaluation

1. Take your best VLM and generate responses on 10 held-out CLEVR examples. Include the image, the question, the ground-truth answer, and your model's generation. Pick a mix of correct and incorrect cases. For each incorrect case, hypothesize whether the failure is in the encoder (image not well understood) or the decoder (language component misinterpreting the question). How would you design an experiment to distinguish between these two failure modes?

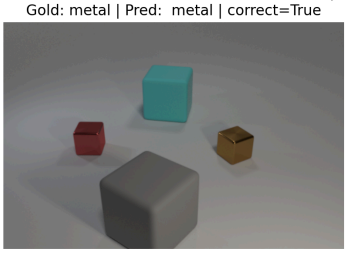
Solution:



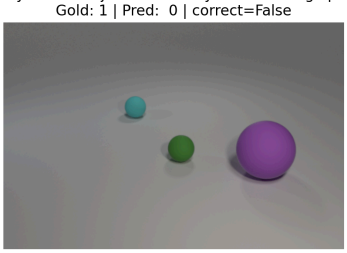
Q: There is a small matte ball in front of the big brown sphere; is it the same color as the block that is behind the small metallic object?
Gold: yes | Pred: yes | correct=True



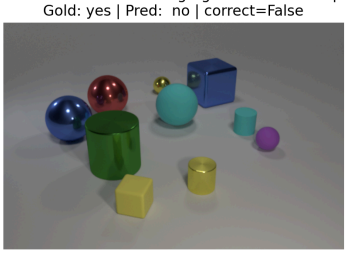
Q: There is a red block that is the same size as the brown block; what is it made of?
Gold: metal | Pred: metal | correct=True



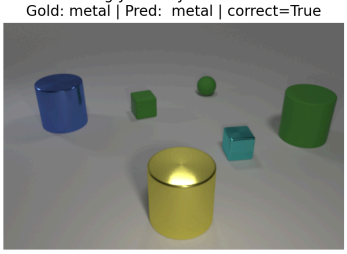
Q: How many rubber objects are red cylinders or large purple balls?
Gold: 1 | Pred: 0 | correct=False



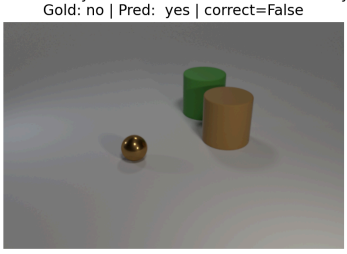
Q: Is there a tiny cyan cylinder to the right of the small metallic thing right of the matte sphere that is behind the cyan matte cylinder?
Gold: yes | Pred: no | correct=False



Q: What is the thing that is both left of the big yellow cylinder and in front of the small matte block made of?
Gold: metal | Pred: metal | correct=True



Q: Is the number of brown matte cylinders less than the number of tiny green metallic blocks?
Gold: no | Pred: yes | correct=False



Q: How many things are either metal objects in front of the large purple cylinder or purple cylinders that are on the left side of the brown thing?
Gold: 4 | Pred: 0 | correct=False

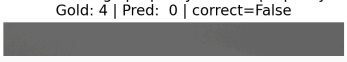


Figure 8: Ten qualitative CLEVR examples from `runs/vlm_qualitative/`.

Question (shortened)	Gold	Prediction	Type	OK?
What size is the yellow thing?	small	small	query_attr	yes
What shape is the green object?	cylinder	sphere	query_attr	no
Same color as block behind metallic object?	yes	yes	spatial	yes
Material of red block same size as brown block?	metal	metal	query_attr	yes
How many rubber objects are red cylinders or large purple balls?	1	0	count	no
Tiny cyan cylinder to the right of ...?	yes	no	spatial	no
Thing left of big yellow cylinder and in front of small matte block material?	metal	metal	spatial	yes
Number of brown matte cylinders $<$ tiny green metallic blocks?	no	yes	count	no
Count of metal objects in front of large purple cylinder or ...	4	0	spatial	no
Material of cube right of object in front of purple thing?	rubber	rubber	spatial	yes

Failures are mixed. Shape confusion (cylinder vs. sphere) and complex counting/spatial questions likely combine encoder and decoder errors: the encoder may not localize all referenced objects, and the decoder may fail on long compositional parses. Attribute/material questions often succeed, suggesting the visual features are sometimes fine but language reasoning breaks on hard queries.

To disentangle encoder vs. decoder failures, I would run controlled probes: (1) give the decoder **oracle** object sets or scene graphs instead of image tokens; (2) ask the vision encoder simpler proxy tasks (count/color probes) on the same images; (3) swap in a stronger frozen decoder while keeping the same visual tokens. If oracle text fixes the answer, the decoder/question understanding is the bottleneck; if simple visual probes fail, the encoder is.

6. Positional Encodings and RoPE

6.1 1D RoPE

1. Verify manually that applying RoPE preserves the norm of each vector (up to numerical precision), and report what you measure.

Solution: Using random inputs and the provided `RoPE1D` / `RoPE2D` modules, I measured:
- RoPE1D max $||\Delta \text{ norm}|| = 9.54 \times 10^{-7}$
- RoPE2D max $||\Delta \text{ norm}|| = 9.54 \times 10^{-7}$

So rotation preserves vector norm up to floating-point precision, as expected.

6.1 Learned PE vs. RoPE in the ViT

1. Retrain CLIP-style on EuroSAT for 20 epochs using (a) learned PE and (b) 1D RoPE. Report zero-shot validation accuracy for each. Then evaluate both models on EuroSAT images upsampled to 96×96 (keeping patch size 8), which produces 144 patches instead of the 64 seen at training. For

the learned-PE baseline, interpolate the learned patch positional embeddings from the 8×8 training grid to the 12×12 evaluation grid. How does each model's accuracy degrade?

Solution:

Positional encoding	64×64 val acc	96×96 val acc	Drop
Learned PE	0.915	0.818	0.097
RoPE 1D	0.907	0.806	0.100
RoPE 2D	0.909	0.821	0.088

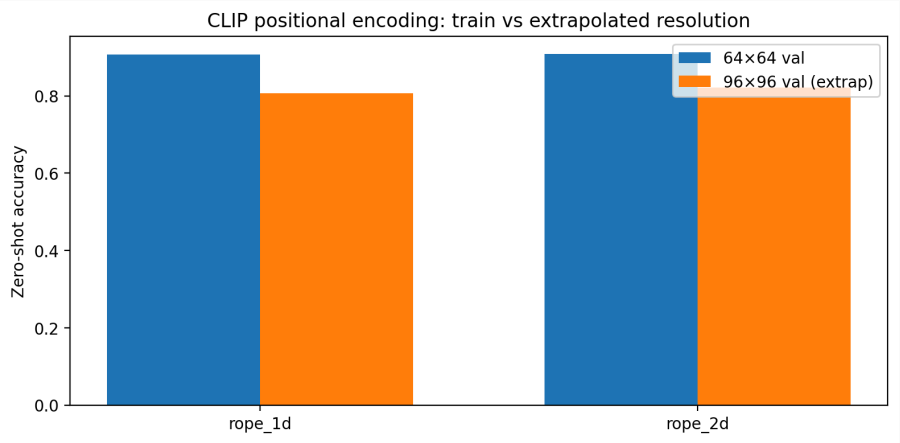


Figure 9: Train-resolution vs. extrapolated-resolution zero-shot accuracy.

All three models degrade at higher resolution, but RoPE—especially 2D RoPE—degrades slightly less than learned PE with interpolation. Learned PE must guess patch positions outside the training grid via interpolation, whereas RoPE applies a principled position-dependent rotation and generalizes more naturally to longer spatial sequences.

6.2 2D RoPE for Image Patches

1. Swap RoPE1D for RoPE2D in your ViT (using each patch's (x, y) grid indices) and re-run the CLIP pretraining + zero-shot evaluation. Does 2D RoPE improve over 1D RoPE on EuroSAT? Include the length-extrapolation test with 2D RoPE as well. Discuss in 2–3 sentences.

Solution: At train resolution, 2D RoPE (0.909) is essentially tied with 1D RoPE (0.907), both slightly below learned PE (0.915). On the 96×96 extrapolation test, however, 2D RoPE (0.821) outperforms 1D RoPE (0.806) and learned PE (0.818). That supports the intuition that encoding (x, y) grid structure helps when patch grids grow beyond the training layout.

6.3 Multimodal RoPE (M-RoPE)

1. What goes wrong with naive 1D position IDs $(0, 1, 2, \ldots)$ when we inject 64 patch tokens plus a CLS token before a 50-token text prompt? Think about (a) position-ID values the decoder was trained on, and (b) the 2D structure of the image.

Solution: Naive 1D IDs assign the first text token position ≈ 65, pushing the entire prompt far beyond the short text sequences the decoder saw during pretraining and making those positions out-of-distribution for its RoPE cache. At the same time, flattening patches into a

single 1D order throws away the 2D neighborhood structure of the image grid, so spatial relations among patches are encoded only indirectly through arbitrary raster order.

2. Under M-RoPE, what position does the first text token get (as a function of the image's grid size)? Why is this choice sensible?

Solution: If the image grid is $G \times G$ patches plus a CLS token, a sensible choice is to start text at temporal index $t = G$ (or one step after the maximum spatial index used by image tokens), while image tokens share a fixed temporal slice and use (x, y) for spatial coordinates. This keeps text positions near the low indices seen during language pretraining instead of jumping to $G^2 + 1$.

3. Why does M-RoPE split the head dimension into three chunks rather than two? What would break if we only used (x, y) and dropped the temporal t?

Solution: Three chunks let RoPE encode modality/temporal progression separately from horizontal and vertical coordinates. If we only used (x, y), text tokens would not have a distinct way to advance a temporal index independently of spatial coordinates, and image vs. text tokens could collide in position space; the model would struggle to distinguish "next word in the prompt" from "different patch coordinate".

6.3 Implementing M-RoPE (Bonus)

1. Implement an M-RoPE-style position assignment for your VLM. Retrain the best configuration from §5 for 1500 steps, using (a) naive 1D position IDs and (b) M-RoPE-style position IDs. Does M-RoPE improve CLEVR accuracy? Does it help more on questions that refer to spatial relations ("left of", "behind", "in front of") than on other questions?

Solution: Not attempted (bonus).